
A11\A11.py

```
import matplotlib.pyplot as plt
import numpy as np
from utils import *

def newton_sqp(f, g, x0, lmb0, tol=1e-7, kmax=100, plot=True):
    """
    Newton-SQP-Verfahren mit Vollschrift zur Minimierung der Funktion f unter Gleichungsnebenbedingungen g

    Input:
    f: Funktion, wobei  $f(x) = (val, grad, hess)$ ,
    g: Liste von Funktionen  $g_1(x), \dots, g_m(x)$ , wobei  $g_i(x) = (val, grad, hess)$ ,
    x0: Vektor (n,) (Startwert für die Iterierten),
    lmb0: Vektor (m,) (Startwert für die Lagrange-Multiplikatoren),
    tol: Skalar (Toleranz, Abbruch wenn die L2-Norm von (deltax, deltalmb)  $\leq$  tol ist),
    kmax: Integer (Anzahl maximaler Iterationen),
    plot: Boolescher Wert (gibt an, ob Daten zur Ploterstellung gespeichert werden sollen).

    Output:
    log: Dictionary vom Verlauf der Optimierung.
    """
    # Dictionary zum Abspeichern der Ergebnisse.
    log = {
        'x0': x0,          # Startwert
        'xsqp': None,      # Lösung des Newton-SQP-Verfahrens (Iterierte)
        'lmbsqp': None,    # Zur Lösung zugehörige Lagrange-Multiplikatoren
        'ksqp': None,     # Anzahl benötigter Iterationen
        'x_list': [],      # Liste der Iterierten
        'val_f_list': [],  # Liste des Funktionswerts in den Iterierten
        'g_res_list': [],  # Liste von Residuen der Nebenbedingungen in den Iterierten, d.h.  $|g_1(x)| + \dots$ 
    }

    # TODO: Aufgabenteil 1. Newton-SQP-Verfahren implementieren
    # BEGIN SOLUTION
    xk = x0
    lmbk = lmb0

    # Überprüfen, ob g eine Liste ist, wenn nicht, dann in eine Liste umwandeln
    if not isinstance(g, list):
        g = [g] # Gleichbedeutend mit  $m = 1$ 
        lmbk = [lmbk]

    # Bestimme die Dimensionen
    n = len(xk)
```

```

m = len(g)

for k in range(kmax):
    # Bestimmen die Ableitungen von f
    _, grad_fxk, hess_fxk = f(xk)

    # Beginnen mit der Berechnung von Hk
    Hk = hess_fxk
    # Bestimmen die Ableitungen von g
    # Konstruieren zunächst die Vektoren und Matrizen zum Speichern aufgrund der mehreren Nebenbeding
    jacobi_g = np.zeros((m, n))
    val_gxk = np.zeros(m)
    # Bestimmen fuer jede Nebenbedingung die Ableitungen
    for i in range(m):
        val_gixk, grad_gixk, hess_gixk = g[i](xk)
        val_gxk[i] = val_gixk
        jacobi_g[i,:] = grad_gixk
        # Berechnen Hk
        Hk = Hk + lmbk[i] * hess_gixk

    # Erstellen die Matrix Ak
    Ak = jacobi_g

    # Erstellen die Blockmatrix und den Loesungsvektor
    block_matrix = np.block([[Hk, Ak.T], [Ak, np.zeros((m, m))]])

    loesungsvektor = np.concatenate([-grad_fxk - np.dot(Ak.T, lmbk), - val_gxk])

    # Loesen das Gleichungssystem
    delta_xlmb = np.linalg.solve(block_matrix, loesungsvektor)
    [delta_xk, delta_lmbk] = np.split(delta_xlmb, [n])

    # Logging der Daten fuer den Plot
    if plot:
        log['x_list'].append(xk.copy())
        val_f, _, _ = f(xk)
        log['val_f_list'].append(val_f)
        log['g_res_list'].append(np.sum(np.abs([g[i](xk)[0] for i in range(m)])))

    # Pruefen Abbruchkriterium
    if np.linalg.norm(delta_xk)**2 + np.linalg.norm(delta_lmbk)**2 <= tol:
        log['xsqp'] = xk
        log['lmbsqp'] = lmbk
        log['ksqp'] = k+1
        return log

    # Update
    xk = xk + delta_xk
    lmbk = lmbk + delta_lmbk
else:
    log['xsqp'] = xk
    log['lmbsqp'] = lmbk
    log['ksqp'] = kmax
# END SOLUTION
return log

```

```

def main():
    # TODO: Aufgabenteil 2i. Funktionen für Problem 1 konstruieren
    # Hinweis: Sowohl f als auch g1 kann als quadratische Funktionen dargestellt werden (Implementierung)
    # BEGIN SOLUTION
    # Bestimmen Matrixen und Vektoren für die quadratischen Funktionen
    A1 = 2 * np.array([[ -1, 0], [0, -1]]) # *2 damit das 1/2 ausgeglichen wird
    b1 = np.array([8, 0])
    c1 = 9
    A2 = - A1
    b2 = np.array([0, 0])
    c2 = -1

    f_problem1 = lambda x: quadratic_function(x, A1, b1, c1)
    g1_problem1 = lambda x: quadratic_function(x, A2, b2, c2)
    # END SOLUTION

    # TODO: Aufgabenteil 2i. Implementierung auf Problem 1 testen, Ergebnis ausgeben und Plot erstellen
    # Hinweis: Sowohl f als auch g1 kann als quadratische Funktionen dargestellt werden (Implementierung)
    # BEGIN SOLUTION
    lmb0 = 0
    tol = 1e-10
    kmax = 100
    x0 = [np.array([1.5, 5.0]), np.array([-0.1, 0.0])]
    xstar = np.array([-1.0, 0.0]) # Lösung vom letzten Blatt?
    for x in x0:
        # kopiert/verändert von unten
        log_sq = newton_sq(f=f_problem1, g=g1_problem1, x0=x, lmb0=lmb0, tol=tol, kmax=kmax)
        print(f'----- \n'
              f' SQP auf Problem 1 (Startwert: x0 = {x})\n'
              f'-----')
        print(f"Anzahl benötigter Iterationen: ksqp = {log_sq['ksqp']}
              Gefundene Lösung: xsqp = {np.round(log_sq['xsqp'], 4)}
              Optimale Lagrange Multiplikatoren: lmb = {log_sq['lmbsq'].flatten()}
              ")
        fig = plot(f=f_problem1, g=g1_problem1, log=log_sq,
                  xstar = xstar,
                  title='Optimierungsverlauf bei Problem 1, x0 = ' + str(x))
        fig.show()
    # END SOLUTION

    # Diskussion
    # BEGIN SOLUTION
    print("Abhängig von der Wahl des x0 werden hier die beiden kritischen Punkte gefunden, dabei führt d
          "Startwert jedoch zum Maximierer und lediglich der zweite zum Minimierer unter der Nebenbedingun
    # END SOLUTION

    # TODO: Aufgabenteil 2ii. Implementierung auf Problem 2 testen und Ergebnis ausgeben
    # Hinweis: Sowohl f als auch g1 kann als quadratische Funktionen dargestellt werden (Implementierung)
    # BEGIN SOLUTION
    A3 = 2 * np.array([[100, 0, 0], [0, 25, 0], [0, 0, 100/9]]) # *2 damit das 1/2 ausgeglichen wird
    A4 = np.zeros((3,3))
    b3 = np.array([-1300, -735, -440])
    b4 = np.array([1, 1, -1])

```

```
c3 = 13983.25
c4 = 0
```

```
f_problem2 = lambda x: quadratic_function(x, A3, b3, c3)
g1_problem2 = lambda x: quadratic_function(x, A4, b4, c4)
x0 = [np.array([0.0, 0.0, 0.0]), np.array([6.5, 14.7, 19.8])]
for x in x0:
    # kopiert/verändert von unten
    log_sq = newton_sq(f=f_problem2, g=g1_problem2, x0=x, lmb0=lmb0, tol=tol, kmax=kmax, plot=False)
    print(f'----- \n'
          f' SQP auf Problem 2 (Startwert: x0 = {x})\n'
          f'-----')
    print(f"""          Anzahl benötigter Iterationen: ksq = {log_sq['ksq']}
    Gefundene Loesung: xsq = {np.round(log_sq['xsq'], 4)}
    Optimale Lagrange Multiplikatoren: lmb = {log_sq['lmbsq'].flatten()}
    """)
```

```
# END SOLUTION
```

```
# Diskussion
```

```
# BEGIN SOLUTION
```

```
print("In diesem Fall führen beide Startwerte zu der gleichen Lösung, welche auch die berechnete Lösung
      "Aufgabe 1 ist. Interessant dabei ist, dass trotz der deutlich unterschiedlichen Distanz zwischen
      "Startwerten und dem Minimierer wird in beiden Fällen der gleiche Lagrange-Multiplikator und\n"
      "auch die gleiche Anzahl an Iterationen benötigt.\n")
```

```
# END SOLUTION
```

```
# TODO: Aufgabenteil 3. Implementierung von newton_sq auf Problem 1 mit mehreren Nebenbedingungen erw
# Hier ist nichts zu tun - Code testet Implementierung automatisch
# Es könnte allerdings notwendig sein, nachträglich in obigen Funktionsaufrufen g1 durch [g1] zu er
```

```
try:
```

```
    lmb0 = np.array([0, 0])
    x0 = np.array([1.5, 5.0])
    xstar = np.array([3/4, np.sqrt(7)/4])
    g2_problem1 = lambda x: g1_problem1(x-np.array([1.5, 0]))
    log_sq = newton_sq(f=f_problem1, g=[g1_problem1, g2_problem1], x0=x0, lmb0=lmb0)
    print(f'----- \n'
          f' SQP auf Problem 1 mit 2 Nebenbedingungen \n'
          f'-----')
    print(f"""          Anzahl benötigter Iterationen: ksq = {log_sq['ksq']}
    Gefundene Lösung: xsq = {np.round(log_sq['xsq'], 4)}
    Residuum: ||xsq - x*||^2 = {np.linalg.norm(log_sq['xsq'] - xstar)}
    Optimale Lagrange Multiplikatoren: lmb = {log_sq['lmbsq'].flatten()}
    """)
```

```
try:
```

```
    fig = plot(f=f_problem1, g=[g1_problem1, g2_problem1], log=log_sq, xstar=xstar,
               title='Optimierungsverlauf bei 2 Nebenbedingungen')
```

```
    fig.show()
```

```
except:
```

```
    print('Implementierung funktioniert für mehr als eine Nebenbedingung, allerdings nicht im Plot')
```

```
except:
```

```
    print('Die Implementierung funktioniert leider nicht für mehr als eine Nebenbedingung :(')
```

```
if __name__ == '__main__':
    main()
```

#TERMINAL OUTPUT:

SQP auf Problem 1 (Startwert: $x_0 = [1.5 \ 5. \]$)

Anzahl benoetigter Iterationen: $ksqp = 9$

Gefundene Loesung: $xsqp = [\ 1. \ -0.]$

Optimale Lagrange Multiplikatoren: $lmb = [-2.99999986]$

SQP auf Problem 1 (Startwert: $x_0 = [-0.1 \ 0. \]$)

Anzahl benoetigter Iterationen: $ksqp = 8$

Gefundene Loesung: $xsqp = [-1. \ 0.]$

Optimale Lagrange Multiplikatoren: $lmb = [5.00000001]$

Abhängig von der Wahl des x_0 werden hier die beiden kritischen Punkte gefunden, dabei führt der erste Startwert jedoch zum Maximierer und lediglich der zweite zum Minimierer unter der Nebenbedingung.

SQP auf Problem 2 (Startwert: $x_0 = [0. \ 0. \ 0.]$)

Anzahl benoetigter Iterationen: $ksqp = 2$

Gefundene Loesung: $xsqp = [\ 6.4 \ 14.3 \ 20.7]$

Optimale Lagrange Multiplikatoren: $lmb = [20.]$

SQP auf Problem 2 (Startwert: $x_0 = [\ 6.5 \ 14.7 \ 19.8]$)

Anzahl benoetigter Iterationen: $ksqp = 2$

Gefundene Loesung: $xsqp = [\ 6.4 \ 14.3 \ 20.7]$

Optimale Lagrange Multiplikatoren: $lmb = [20.]$

In diesem Fall führen beide Startwerte zu der gleichen Lösung, welche auch die berechnete Lösung aus Aufgabe 1 ist. Interessant dabei ist, dass trotz der deutlich unterschiedlichen Distanz zwischen den Startwerten und dem Minimierer wird in beiden Fällen der gleiche Lagrange-Multiplikator und auch die gleiche Anzahl an Iterationen benötigt.

SQP auf Problem 1 mit 2 Nebenbedingungen

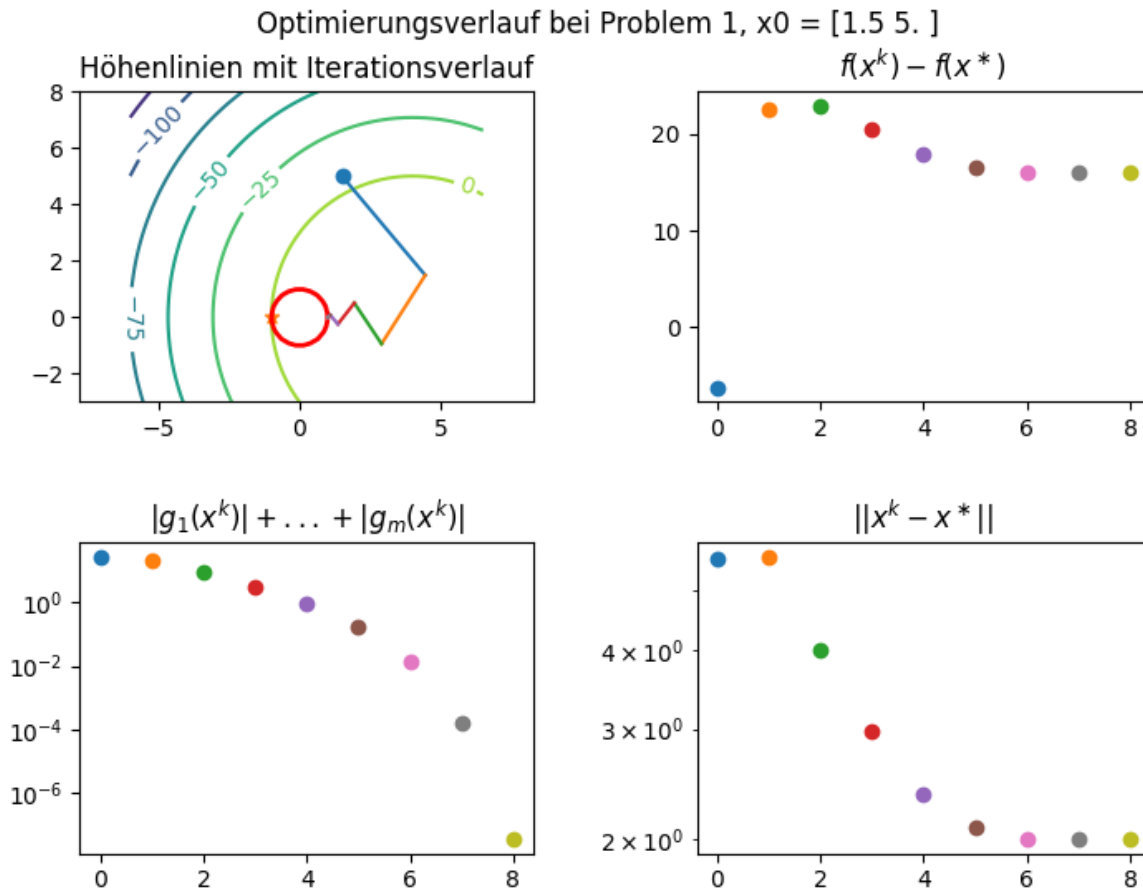
Anzahl benötigter Iterationen: ksqp = 7

Gefundene Lösung: xsqp = [0.75 0.6614]

Residuum: $\|x_{sqp} - x^*\|^2 = 7.794337486544123e-08$

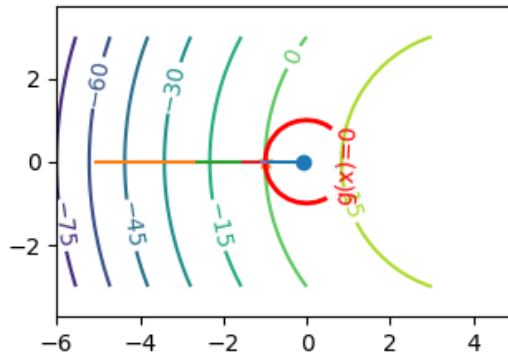
Optimale Lagrange Multiplikatoren: lmb = [-1.66666679 2.66666654]

#PLOTS:

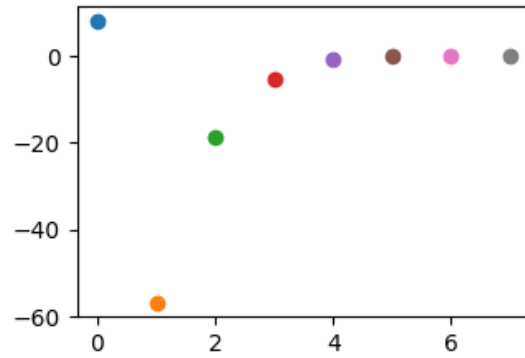


Optimierungsverlauf bei Problem 1, $x_0 = [-0.1 \ 0.]$

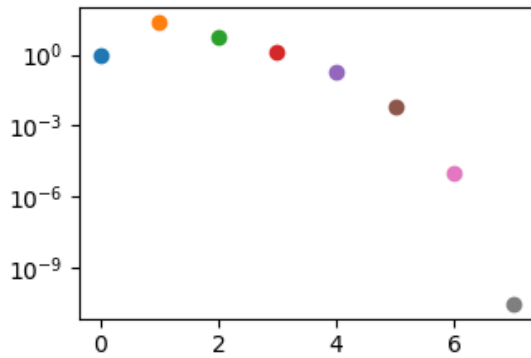
Höhenlinien mit Iterationsverlauf



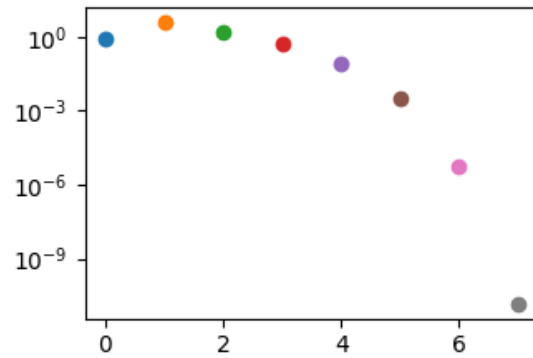
$f(x^k) - f(x^*)$



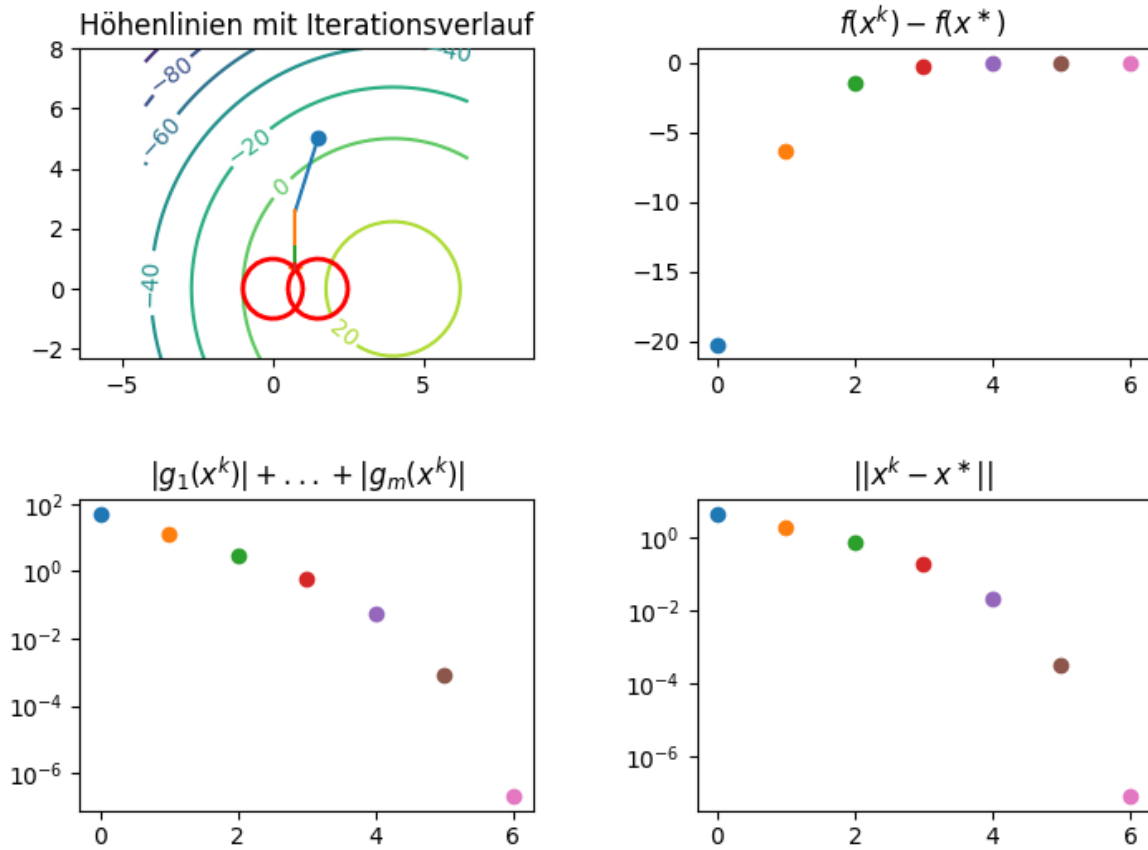
$|g_1(x^k)| + \dots + |g_m(x^k)|$



$\|x^k - x^*\|$



Optimierungsverlauf bei 2 Nebenbedingungen



```

import numpy as np
import matplotlib.pyplot as plt

# Quadratische Funktion, angepasst von Blatt 5. ACHTUNG: Hier  $x.T @ A @ x + b.T @ x + c \rightarrow$  positiver linearer
def quadratic_function(x, A, b, c):
    """
    Quadratische Funktion

    Input:
        x: Variable,
        A: Matrix,
        b: Vektor,
        c: Skalar.

    Output:
        val: Skalar ( $f(x)$ ),
        grad: Vektor ( $\nabla f(x)$ ),
        hess: Matrix ( $\nabla^2 f(x)$ ).
    """
    # Note: The ndarray x must have size (n,) for one coordinate reps. (m,n) for multiple coordinates
    val = 0.5 * np.einsum('...i,...i->...', x @ A, x) + np.inner(b, x) + c
    grad = x @ A + b # Note: x @ A instead of A @ x in order to deal with multiple coordinates
    hess = A
    return val, grad, hess

# Plot-Funktion, übernommen vom Blatt 5
def plot(f, g, log, xstar, title):
    """
    Funktion zur Erstellung von Plots zum Optimierungsverlauf anhand der Ergebnisse in log.
    Es werden Höhenlinien und Iterationsverlauf auf einem 2D-Gitter sowie Konvergenzeigenschaften dargestellt.
    Die Funktion liefert fig zur Ansicht. Stellen Sie diese dar, indem Sie plt.show() in Ihrer Hauptdatei aufrufen.

    Input:
        f: Optimierte Funktion
        log: Dictionary, das mindestens folgende Einträge enthält:
            log = {
                'x0': Vektor, # Startwert
                'x_list': Liste, # Iterierte
                'val_f_list': Liste, # Funktion ausgewertet an den Iterierten
                'g_res_list': Liste # Residuen der Nebenbedingungen in den Iterierten in 1-Norm
            }
        xstar: Vektor, tatsächliche Lösung des Optimierungsproblems
        title: Titel des Plots.

    Output:
        fig: hergestellte Figure mit den Plots
    """
    # In dieser Funktion initialisieren wir die Figure mit den Subplots
    def initialize_figure():
        fig, axes = plt.subplots(nrows=2, ncols=2)
        fig.set_size_inches(8, 6)
        fig.tight_layout(pad=4.0)

```

```

fig.suptitle(title)
return fig, axes

# Berechne Höhenlinien
def hoehen_linien(fctn):
    # Definitionsbereich wählen (x0 und xstar helfen abzuschätzen, wie groß dieser sein muss)
    n = 50
    x0 = log['x0']
    xaxis = np.linspace(min(x0[0], xstar[0]) - 5, max(x0[0], xstar[0]) + 5, n)
    yaxis = np.linspace(min(x0[1], xstar[1]) - 3, max(x0[1], xstar[1]) + 3, n)

    # Gitterpunkte und Funktionswerte auf den Gitterpunkten
    xx, yy = np.meshgrid(xaxis, yaxis, sparse=False, indexing='ij')
    zz, _, _ = fctn(np.concatenate((xx.reshape(n * n, 1), yy.reshape(n * n, 1)), axis=-1))
    zz = zz.reshape(n, n)
    return xx, yy, zz

# Initialisiere Abbildung
fig, ax = initialize_figure()

# Plotten der Höhenlinien
ax[0, 0].set_title('Höhenlinien mit Iterationsverlauf')
ax[0, 0].set_aspect('equal', 'datalim') # Um die Orthogonalität der Suchrichtungen besser zu visuali

xx, yy, zzf = hoehen_linien(f)
x0 = log['x0']
x_list = log['x_list']
CSf = ax[0, 0].contour(xx, yy, zzf)
ax[0, 0].clabel(CSf, inline=1, fontsize=10)

if isinstance(g, list):
    for i, gi in enumerate(g):
        _, _, zzg = hoehen_linien(gi)
        CSg = ax[0, 0].contour(xx, yy, zzg, levels=[0], colors='red', linewidths=2)
        ax[0, 0].clabel(CSg, inline=True, fmt=r'$g_{i}(x)=0$', fontsize=10)
else:
    _, _, zzg = hoehen_linien(g)
    CSg = ax[0, 0].contour(xx, yy, zzg, levels=[0], colors='red', linewidths=2)
    ax[0, 0].clabel(CSg, inline=True, fmt='g(x)=0', fontsize=10)

ax[0, 0].scatter(*x0, marker='o')
ax[0, 0].scatter(*xstar, marker='*')
for k in range(len(x_list)-1):
    xk = x_list[k]
    xkplus1 = x_list[k+1]
    ax[0, 0].plot([xk[0], xkplus1[0]], [xk[1], xkplus1[1]])

# Plotten der Zielfunktionsfehler
ax[0, 1].set_title(r'$f(x^k) - f(x^{\ast})$')
val_list = log['val_f_list']
val_star = f(xstar)[0]
for k, valk in enumerate(val_list):
    ax[0, 1].scatter(k, valk - val_star)

# Plotten der Residuen der Gleichungsnebenbedingungen
ax[1, 0].set_title(r'$|g_1(x^k)| + \dots + |g_m(x^k)|$')

```

```

g_res_list = log['g_res_list']
for k, g_res_k in enumerate(g_res_list):
    ax[1, 0].scatter(k, g_res_k)
ax[1, 0].set_yscale('log')

# Plotten der Konvergenz der Iterierten
ax[1, 1].set_title(r'$||x^k - x^{ast}||$')
x_list = log['x_list']
for k, xk in enumerate(x_list):
    ax[1, 1].scatter(k, np.linalg.norm(xk - xstar))
ax[1, 1].set_yscale('log')

return fig

```

#TERMINAL OUTPUT:

#PLOTS: